Машинное обучение

Метод градиентного спуска 1-го порядка

Команда 1: Матвейчик Екатерина, Корся Ева

План презентации

1) Постановка задачи оптимизации в ML
2) Математическая основа и принцип работы метода
3) Ограничения на используемую функцию
4) Выбор шага (learning rate)
5) Преимущества и недостатки метода
6) Численный пошаговый пример
7) Программная реализация на Python
8) Модификации: Batch, SGD, Mini-batch
9) Модификации: Adam, Momentum
10) Сравнение с альтернативными методами

Постановка задачи оптимизации в ML

Любое обучение модели сводится к поиску её идеальных настроек:

w (веса/weights): Настраиваемые параметры модели (например, коэффициенты w_0 и w_1 в уравнении прямой $\hat{y} = w_0 + w_1 x$)

$L(w)$ (функция потерь/ Loss): Величина ошибки модели при текущих весах w.

ЦЕЛЬ: Найти параметры w^*, при которых величина ошибки минимальна

$$w^* = \arg\min_w L(w)$$

Почему не аналитически?

- Аналитическое решение (как в МНК для линейной регрессии) существует редко
- Функции потерь в нейросетях сильно нелинейны

Решение: численные итеративные методы

Математическая основа метода

Градиент - вектор частных производных функции по всем параметрам:

$$\nabla L(w) = \left(\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \dots, \frac{\partial L}{\partial w_n} \right)^T$$

Свойство градиента: Всегда указывает направление наискорейшего возрастания функции

Антиградиент $-\nabla L(w)$: Указывает направление наискорейшего убывания

Градиент на реальных данных

В ML функция ошибки – это среднее по всей обучающей выборке из N объектов:

$$L(w) = \frac{1}{N}\sum_{i=1}^{N}(w \cdot x_i - y_i)^2$$

Здесь x_i – признак (входное значение) для i-го объекта (например, площадь квартиры), y_i— истинный целевой ответ для i-го объекта из обучающей выборки (например, реальная стоимость этой квартиры), $w \cdot x_i$(или $\hat{y}_i$) — предсказание линейной модели для i-го объекта.

Берем производную по весу w и получаем градиент:

$$\nabla L(w) = \frac{2}{N}\sum_{i=1}^{N}(w \cdot x_i - y_i) \cdot x_i$$

Формула обновления весов (Шаг спуска)

Итеративное правило обновления параметров:

$$w_{t+1} = w_t - \alpha \cdot \nabla L(w_t)$$

Где:

w_t— вектор параметров на текущем шаге t.

w_{t+1}— обновленный вектор параметров.

$\nabla L(w_t)$— градиент функции потерь в точке w_t.

α(Learning Rate) — шаг обучения, скаляр $\alpha > 0$.

Ограничения на используемую функцию

Для корректной работы метода функция потерь $L(w)$ должна удовлетворять строгим математическим критериям:

1. Дифференцируемость: Функция должна иметь производную (градиент) в каждой точке. (Для негладких функций используют субградиенты).

2. Непрерывность: Отсутствие разрывов.

3. Гладкость (Липшицева непрерывность градиента): Скорость изменения градиента должна быть ограничена.

$$||\nabla L(x) - \nabla L(y)|| \leq M||x - y||$$

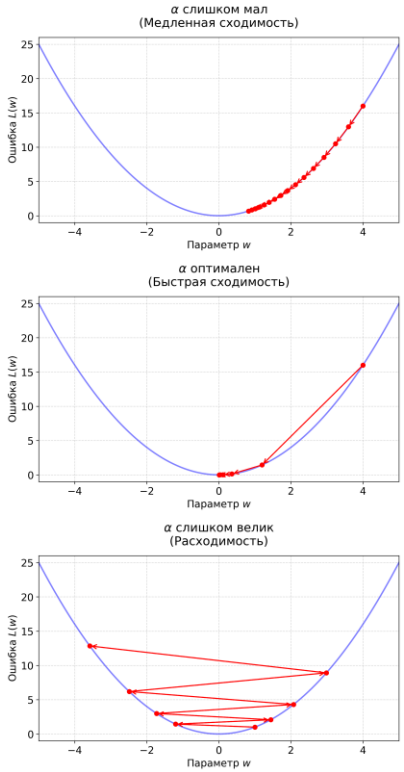
Важность гиперпараметра α (Learning Rate)

Влияние размера шага α на сходимость:

α **слишком мал**: Сходимость очень медленная. Алгоритм требует тысяч лишних итераций.

α **оптимален**: Быстрое и плавное достижение минимума.

α **слишком велик**: Алгоритм перепрыгивает минимум. Ошибка начинает возрастать, веса уходят в бесконечность.



Выбор оптимального шага α

Следствие из условия Липшица (Descent Lemma): Если градиент функции является M-липшицевым (максимальная кривизна ограничена M), функцию можно мажорировать параболой. Для любых точек x и y выполняется квадратичная верхняя оценка:

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{M}{2} \|y - x\|^2$$

Подстановка шага спуска: Делаем шаг в направлении антиградиента $y = x_k - \alpha \nabla f(x_k)$. Получаем оценку убывания:

$$f(x_{k+1}) \leq f(x_k) - \alpha \left(1 - \frac{M\alpha}{2}\right) \|\nabla f(x_k)\|^2$$

Условие монотонности:

$$1 - \frac{M\alpha}{2} \geq 0 \Rightarrow \alpha \leq \frac{2}{M}$$

Выбор оптимального шага α

Сходимость	Теоретическое условие	Смысл
Идеальный шаг	$\alpha \leq \frac{1}{M}$	Гарантированное строгое монотонное убывание функции
Сходится	$\alpha < \frac{2}{M}$	Достижение минимума ($\|\nabla f\| \to 0$), но возможны колебания
Расходится	$\alpha \geq \frac{2}{M}$	Алгоритм расходится, траектория перепрыгивает минимум

Проблема выпуклости функции (convexity)

Классификация функций по выпуклости:

Выпуклая функция (Convex): Имеет только один глобальный минимум. Градиентный спуск ГАРАНТИРОВАННО находит оптимум (пример: Линейная регрессия, Логистическая регрессия).

Невыпуклая функция (Non-convex): Сложный ландшафт. Множество локальных минимумов, седловых точек и плато (пример: Глубокие нейронные сети).

Риск: В невыпуклых функциях классический градиентный спуск может навсегда застрять в локальном минимуме.

Преимущества и недостатки метода

Преимущества:

Легкость в понимании и программной реализации.

Низкие требования к памяти — нужно хранить только веса и их градиенты (O(N) памяти).

Эффективно масштабируется на модели с миллиардами параметров (LLM).

Недостатки:

Риск застревания в локальных минимумах и седловых точках.

Медленное прохождение "плато" (участков с почти нулевым градиентом).

Сильная зависимость результата от выбора стартовой точки и шага α.

Требует масштабирования признаков (нормирование и центрирование), иначе спуск будет зигзагообразным. Для каждого значения признака x: $x_{new} = \frac{x-\mu}{\sigma}$, где μ– среднее значение признака по выборке, σ– стандартное отклонение

Пошаговый численный пример

Функция потерь и её градиент для нашего примера:

$$L(w) = w^2;\ \nabla L(w) = 2w$$

Проверка условия Липшица:
$||\nabla L(x) - \nabla L(y)|| = ||2x - 2y|| = 2||x - y||$.
Отсюда получаем, что константа Липшица M = 2.

Классический шаг метода:

$$w_{t+1} = w_t - \alpha \nabla L(w_t)$$

Подставим значение градиента:

$$w_{t+1} = w_t - \alpha(2w_t) = w_t(1 - \alpha) = w_t q$$

Получили множитель q, который полностью определяет, как быстро и по какой траектории алгоритм придет к минимуму.

Анализ разных шагов обучения α и соответствующего множителя q:

Шаг α	Множитель q = 1 - 2α	Поведение траектории
0.2 (<1/M)	0.6	Монотонная сходимость (плавное приближение)
0.5 (=1/M)	0.0	Сходимость за 1 шаг (мгновенное попадание в ноль)
0.9 (<2/M)	-0.8	Сходимость с колебаниями (прыжки со сменой знака)
1.0 (=2/M)	-1.0	Незатухающие колебания (бесконечный цикл)
1.1 (>2/M)	-1.2	Расходимость (ошибка уходит в бесконечность)

Пошаговый численный пример

Начальные условия для запуска алгоритма:
Стартовая точка $w_0 = 5$.
Выбранный шаг обучения $\alpha = 0.2$. Точное решение $w^* = 0$.

Множитель за итерацию при таком шаге составляет: $q = 1 - 2(0.2) = 0.6$

Расчет 6 итераций по формуле обновления веса $w_{t+1} = w_t q$:
Итерация 1: $w_1 = 5 \times 0.6 = 3.0$
Итерация 2: $w_2 = 3.0 \times 0.6 = 1.8$
Итерация 3: $w_3 = 1.8 \times 0.6 = 1.08$
Итерация 4: $w_4 = 1.08 \times 0.6 = 0.648$
Итерация 5: $w_5 = 0.648 \times 0.6 = 0.3888$
Итерация 6: $w_6 = 0.3888 \times 0.6 = 0.23328$

Итог: алгоритм демонстрирует монотонную сходимость к точке 0, так как множитель q находится в безопасном диапазоне от 0 до 1.

Численный пример на датасете

Задача: Оценить стоимость квартиры (y) в зависимости от её площади (x). Для простоты используем модель линейной регрессии без свободного коэффициента: $\hat{y} = w \cdot x$.

Обучающая выборка ($N = 3$ объекта):

Объект (i)	Площадь x	Реальная цена y (млн руб.)
Квартира 1	3 (30 кв. м)	10
Квартира 2	4 (40 кв. м)	13
Квартира 3	5 (50 кв. м)	17

Стартовый вес модели $w_0 = 2.0$. Сейчас модель полагает, что 10 кв.м стоят 2 млн рублей.

Шаг обучения (learning rate) зададим как $\alpha = 0.02$.

Фактическая цена за 10 кв.м: Квартира 1: 10 / 3 = 3.33 млн, Квартира 2: 13 / 4 = 3.25 млн, Квартира 3: 17 / 5 = 3.40 млн

Цель градиентного спуска – найти компромиссный вес, минимизирующий суммарную ошибку MSE. Найдем точное решение аналитически (через формулу МНК):

$$w^* = \frac{\sum x_i y_i}{\sum x_i^2}$$

Подставляем значения площади и цены из датасета: $w^* = \frac{3 \cdot 10 + 4 \cdot 13 + 5 \cdot 17}{3^2 + 4^2 + 5^2} = \frac{30 + 52 + 85}{9 + 16 + 25} = \frac{167}{50} = 3.34$

Итог: Глобальный минимум функции ошибки находится в точке 3.34. Идеальная рыночная цена для нашего датасета – 3.34 млн за 10 кв.м.

Численный пример на датасете (1-ая итерация)

Формула градиента ошибки (MSE) для всей выборки:

$$\nabla L = \frac{2}{N}\sum_{i=1}^{N}(wx_i - y_i)\cdot x_i$$

Считаем локальный градиент для каждой квартиры при текущем весе $w_0 = 2.0$:

Усредняем градиент по всей выборке:

$$\nabla L = \frac{2}{3}\cdot(-12-20-35) = \frac{2}{3}\cdot(-67) \approx -44.67$$

Обновляем вес по формуле шага (шаг $\alpha = 0.02$):

$$w_1 = 2.0 - 0.02\cdot(-44.67) = 2.0 + 0.893 = 2.893$$

Объект	x_i	y_i	Прогноз w_0x_i	Ошибка $w_0x_i - y_i$	Градиент $(w_0x_i - y_i)x_i$
Квартира 1	3	10	6	-4	$-4\times 3 = -12$
Квартира 2	4	13	8	-5	$-5\times 4 = -20$
Квартира 3	5	17	10	-7	$-7\times 5 = -35$

Численный пример на датасете (2-ая итерация)

Текущий вес после первого шага: $w_1 = 2.893$. Считаем градиенты заново:

Объект	x_i	y_i	Прогноз $w_1 x_i$	Ошибка $w_1 x_i - y_i$	Градиент $(w_1 x_i - y_i) x_i$
Квартира 1	3	10	8.679	-1.321	$-1.321 \times 3 = -3.963$
Квартира 2	4	13	11.572	-1.428	$-1.428 \times 4 = -5.712$
Квартира 3	5	17	14.465	-2.535	$-2.535 \times 5 = -12.675$

Считаем итоговый градиент и обновляем вес:

$$\nabla L = \frac{2}{3} \cdot (-3.963 - 5.712 - 12.675) = \frac{2}{3} \cdot (-22.35) = -14.9$$

$$w_2 = 2.893 - 0.02 \cdot (-14.9) = 2.893 + 0.298 = 3.191$$

Итог: За две итерации градиентный спуск достиг значения 3.191 (идеальный оптимум 3.34)

Программная реализация на Python. Numpy

```python
import numpy as np

x, y = np.array([3, 4, 5]), np.array([10, 13, 17])
w, alpha = 2.0, 0.02

for i in range(10):
    error = (w * x) - y
    grad = (2 / len(x)) * np.sum(error * x)

    w -= alpha * grad  # Математический шаг спуска

    print(f"Итерация {i + 1}: Новый вес = {w:.3f}")
```

```
Итерация 1: Новый вес = 2.893
Итерация 2: Новый вес = 3.191
Итерация 3: Новый вес = 3.290
Итерация 4: Новый вес = 3.323
Итерация 5: Новый вес = 3.334
Итерация 6: Новый вес = 3.338
Итерация 7: Новый вес = 3.339
Итерация 8: Новый вес = 3.340
Итерация 9: Новый вес = 3.340
Итерация 10: Новый вес = 3.340
```

Критерий остановки

На практике для выхода из цикла используют комбинацию следующих условий (срабатывает то, которое наступит первым):

1. Исчерпан лимит итераций (Max Epochs). Самый простой и обязательный предохранитель. Задается жесткое число шагов (например, max_iter = 1000).

2. Градиент близок к нулю (Gradient Norm Tolerance). Математическое условие достижения локального минимума или плато: $\|\nabla L(w_t)\| < \epsilon$

3. Стабилизация функции потерь (Loss Tolerance). Значение ошибки практически перестало меняться между соседними шагами алгоритма: $|L(w_t) - L(w_{t-1})| < \epsilon$

4. Стабилизация весов (Weight Tolerance). Сами параметры модели больше не сдвигаются: $\|w_t - w_{t-1}\| < \epsilon$

Программная реализация на Python. Pytorch (1)

```python
import torch
import math
import matplotlib.pyplot as plt

# 1. Подготовка данных (Синусоида)
dtype = torch.float
device = torch.device("cpu")
x = torch.linspace(-math.pi, math.pi, 2000, device=device, dtype=dtype)
y = torch.sin(x)

# 2. Инициализация весов полинома (a, b, c, d)
weights = torch.randn(4, device=device, dtype=dtype, requires_grad=True)
learning_rate = 1e-6
prev_loss = float('inf')

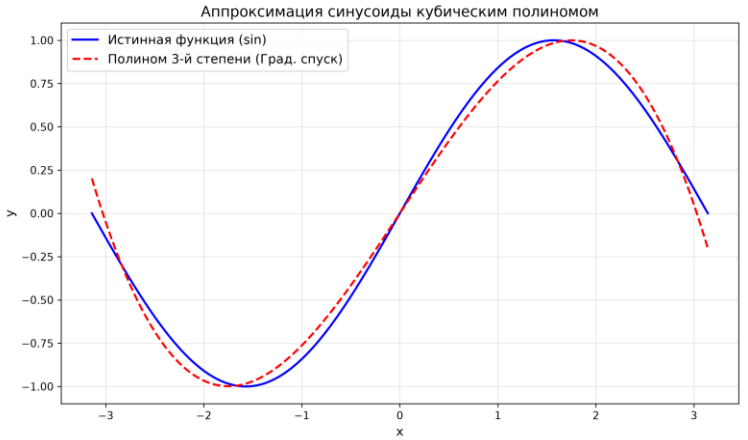
# 3. Цикл обучения (Градиентный спуск с Autograd)
for t in range(10000):
    y_pred = weights[0] + weights[1] * x + weights[2] * (x ** 2) + weights[3] * (x ** 3)
    loss = (y_pred - y).pow(2).sum()
    loss.backward()

    with torch.no_grad():
        weights -= learning_rate * weights.grad
        weights.grad.zero_()
```

Программная реализация на Python. Pytorch (2)

```
26      if (t+1) % 500 == 499:
27          print(f"Эпоха {t + 1}, Ошибка: {loss.item():.6f}")
28
29      # Критерий остановки (Loss Tolerance)
30      if abs(prev_loss - loss.item()) < 1e-8:
31          print(f"Остановка на эпохе {t + 1}. Ошибка стабилизировалась: {loss.item():.6f}")
32          break
33      prev_loss = loss.item()
34
35  print(f"Полином: y = {weights[0].item():.3f} + {weights[1].item():.3f}x + {weights[2].item():.3f}x^2 + {weights[3].item():.3f}x^3")
```

```
Эпоха 499, Ошибка: 113.827393
Эпоха 999, Ошибка: 27.442194
Эпоха 1499, Ошибка: 12.121104
Эпоха 1999, Ошибка: 9.403326
Эпоха 2499, Ошибка: 8.921166
Эпоха 2999, Ошибка: 8.835621
Эпоха 3499, Ошибка: 8.820442
Эпоха 3999, Ошибка: 8.817749
Остановка на эпохе 4003. Ошибка стабилизировалась: 8.817741
Полином: y = 0.001 + 0.857x + -0.000x^2 + -0.093x^3
```



Оптимизация строго выпуклой задачи (1)

Переход к матричным вычислениям: Демонстрация работы градиентного спуска на многомерном синтетическом датасете (бинарная классификация двух пересекающихся классов).

Архитектура модели: Однослойная сеть (один линейный слой + нелинейная функция активации сигмоида).

Математическая модель: В качестве функции потерь используется бинарная кросс-энтропия (Binary Cross-Entropy)

Функционал ошибки (BCE Loss):

$$L(w) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

Прогноз модели (вероятность принадлежности к классу):

$$\hat{y}_i = \sigma(w^T x_i + b)$$

Оптимизация строго выпуклой задачи (2)

```python
# 1. Генерация датасета (два класса с небольшим пересечением)
X_np, y_np = make_blobs(n_samples=1000, centers=2, n_features=2,
                        random_state=42, cluster_std=1.5)

# Конвертация в тензоры PyTorch
X = torch.tensor(X_np, dtype=torch.float32)
y = torch.tensor(y_np, dtype=torch.float32).view(-1, 1)


# 2. Определение модели (Логистическая регрессия = 1 слой + Сигмоида)
class LogisticRegressionModel(nn.Module):  2 usages
    def __init__(self):
        super(LogisticRegressionModel, self).__init__()
        self.linear = nn.Linear(2, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        return self.sigmoid(self.linear(x))
```

```python
model = LogisticRegressionModel()

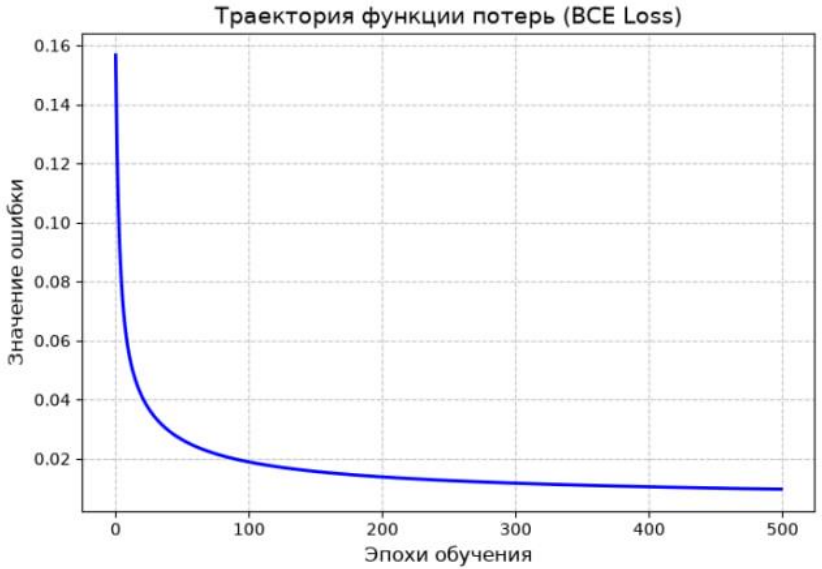
# 3. Настройки градиентного спуска
criterion = nn.BCELoss()  # Binary Cross Entropy (строго выпуклая функция)
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

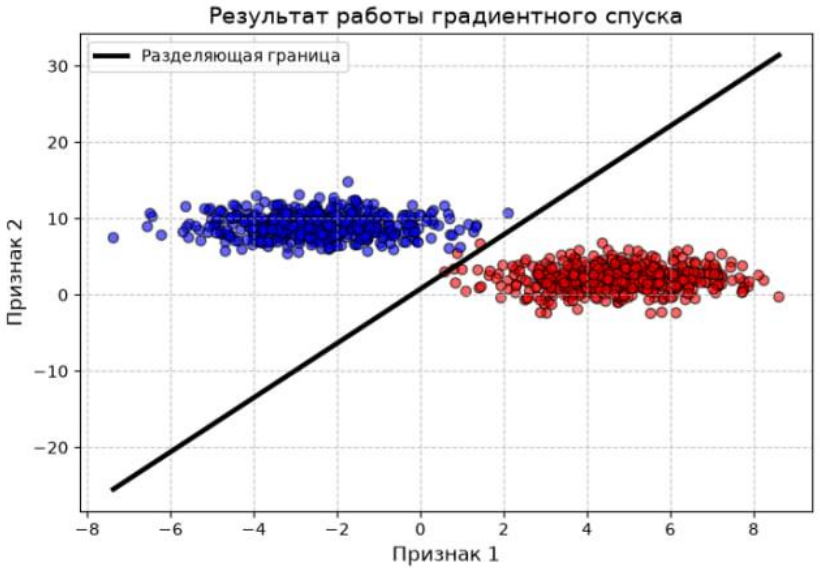
# 4. Цикл обучения
epochs = 500
loss_history = []

for epoch in range(epochs):
    # Прямой проход (forward)
    y_predicted = model(X)
    loss = criterion(y_predicted, y)
    loss_history.append(loss.item())

    # Обратное распространение и шаг градиентного спуска (backward)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Оптимизация строго выпуклой задачи (3)



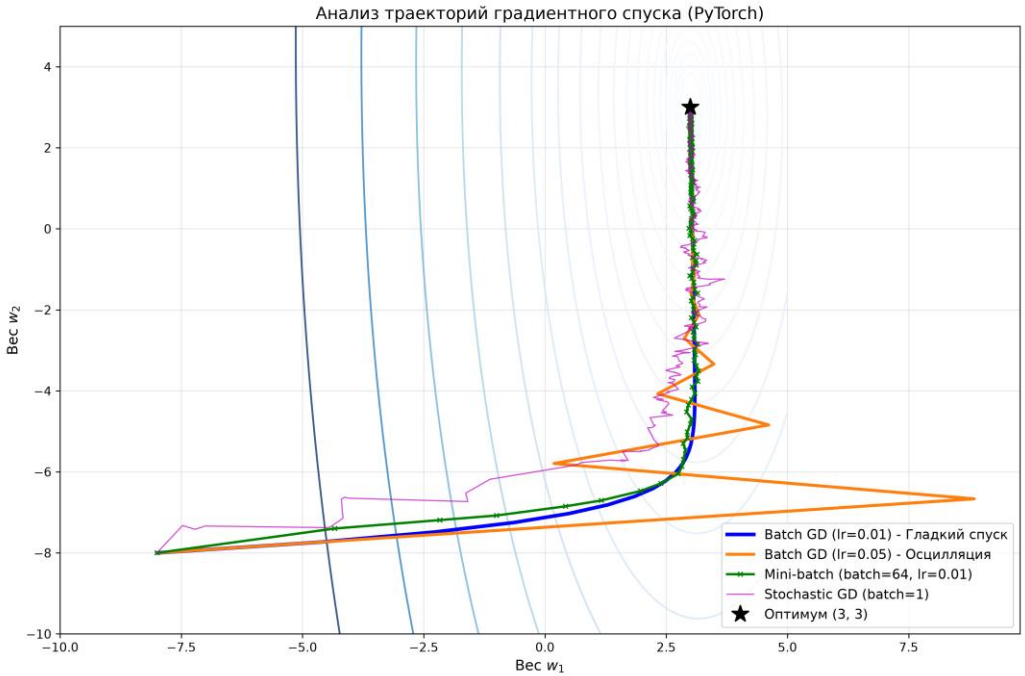


Проблема объема данных: Мини-батчи

Batch (Пакетный спуск).
Суммируем градиент ошибки по всем N объектам датасета: $\nabla L(w) = \frac{1}{N}\sum_{i=1}^{N} \nabla L_i(w)$. *Минус:* Не влезает в оперативную память на реальных больших данных.

Stochastic (Стохастический спуск).
Берем градиент только от одного случайно выбранного объекта i: $\nabla L(w) \approx \nabla L_i(w)$. *Минус:* Очень «шумный» вектор градиента.

Mini-batch (Мини-пакетный).
Данные разбиваются на пакеты ($m = 32, 64, 256$ объектов, m ≪ N). $\nabla L(w) \approx \frac{1}{m}\sum_{i=1}^{m} \nabla L_i(w)$.



Проблема ландшафта: Импульс и адаптивный шаг

Проблема: Базовый алгоритм застревает в локальных минимумах и седловых точках.

Momentum (Импульс).

- Вектор шага вычисляется как взвешенная сумма прошлых градиентов.
- Накапливает скорость при стабильном движении в одном направлении.
- Взаимно гасит колебания при частой смене знака градиента, выравнивая траекторию.

Adam (Adaptive Moment Estimation).Текущий стандарт в Deep Learning.

- Рассчитывает индивидуальный размер шага обучения (α) для каждого отдельного параметра.
- Учитывает первый момент (историю направления) и второй момент (историю квадратов градиентов).
- Автоматически уменьшает шаг для сильно колеблющихся весов и увеличивает для тех, которые обновляются медленно.

Место градиентных методов в теории оптимизации

Методы второго порядка (метод Ньютона, L-BFGS)
Опираются на матрицу Гессе (вторые производные) для построения точной квадратичной аппроксимации
Ограничение: Вычисление и обращение гессиана вычислительно неподъемно для моделей машинного обучения с большим числом параметров.

Методы первого порядка (Градиентный спуск, Momentum, Adam) – *фокус работы*
Используют вектор антиградиента (первые производные).
Преимущество: Идеальный компромисс между скоростью сходимости и затратами оперативной памяти.

Методы нулевого порядка (Метаэвристики)
Дифференциальная эволюция, Генетические алгоритмы, Метод роя частиц (PSO).
Не требуют дифференцируемости, работают напрямую со значениями целевой функции.
Применение: Эффективны для оптимизации сложных топологических структур или на сильно изрезанных ландшафтах, но избыточны и медлительны для гладких функций потерь.

Итоги

- Изучены теоретические основы градиентного спуска, реализована базовая математика алгоритма (NumPy) и индустриальный пайплайн с автоматическим дифференцированием (PyTorch).

- Проведен анализ стратегий батчинга (Batch, Mini-batch, SGD) с программным построением реальных траекторий алгоритмов на рельефе функции потерь.

- Рассмотрены проблемы сходимости алгоритма в сложных ландшафтах и проанализирована концептуальная механика продвинутых оптимизаторов (Momentum, Adam) для их решения.

Спасибо за внимание!